

Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Российский экономический университет имени Г.В. Плеханова»
Пермский филиал

ПРАКТИЧЕСКАЯ РАБОТА №2

по дисциплине

«Поддержка и тестирование программных модулей»

Тема: Разработка приложения «Калькулятор»

Выполнил: студент группы ИПс-11

Бахматов М.Б.

Проверил: Берестов Д.Б.

Пермь, 2026

ОГЛАВЛЕНИЕ

[1. Введение](#)

[2. Используемые технологии](#)

[3. Структура проекта](#)

[4. Создание проекта в Visual Studio](#)

[5. Реализация библиотеки классов](#)

[5.1 Алгоритм вычисления выражений](#)

[5.2 CalculatorEngine](#)

[5.3 Модели](#)

[5.4 Сервисы](#)

[6. Реализация WPF-приложения](#)

[6.1 Архитектура MVVM](#)

[6.2 Интерфейс](#)

[6.3 Возможности приложения](#)

[6.4 Ввод с клавиатуры](#)

[6.5 Сохранение данных](#)

[7. Модульное тестирование](#)

[7.1 Структура тестов](#)

[7.2 Изоляция тестов](#)

[7.3 Результаты тестирования](#)

[8. Публикация приложения](#)

[9. Размещение на GitHub](#)

[10. Заключение](#)

1. Введение

Настоящая практическая работа посвящена разработке настольного приложения «Калькулятор» на платформе .NET 8 с использованием технологии WPF (Windows Presentation Foundation). Работа выполнена в соответствии с заданием по дисциплине «Поддержка и тестирование программных модулей».

Цель работы — создать приложение-калькулятор, реализующее вычисление арифметических выражений с поддержкой операторов $+$, $-$, $*$, $/$, $^$ (возведение в степень) и скобок. Основная логика вынесена в отдельную библиотеку классов; интерфейс реализован в виде WPF-приложения по паттерну MVVM. Для библиотеки написаны модульные тесты.

2. Используемые технологии

В ходе разработки использовались следующие технологии и инструменты:

Компонент	Технология
Платформа	.NET 8
UI-фреймворк	WPF (net8.0-windows)
Библиотека логики	Class Library (net8.0)
Тестирование	xUnit
Сериализация	System.Text.Json
IDE	Visual Studio 2022
Система контроля версий	Git / GitHub

3. Структура проекта

Решение состоит из трёх проектов:

- BakhmatovCalculatorLib — библиотека классов с бизнес-логикой
- BakhmatovCalculatorApp — WPF-приложение (интерфейс)
- BakhmatovCalculatorTests — проект модульных тестов (xUnit)

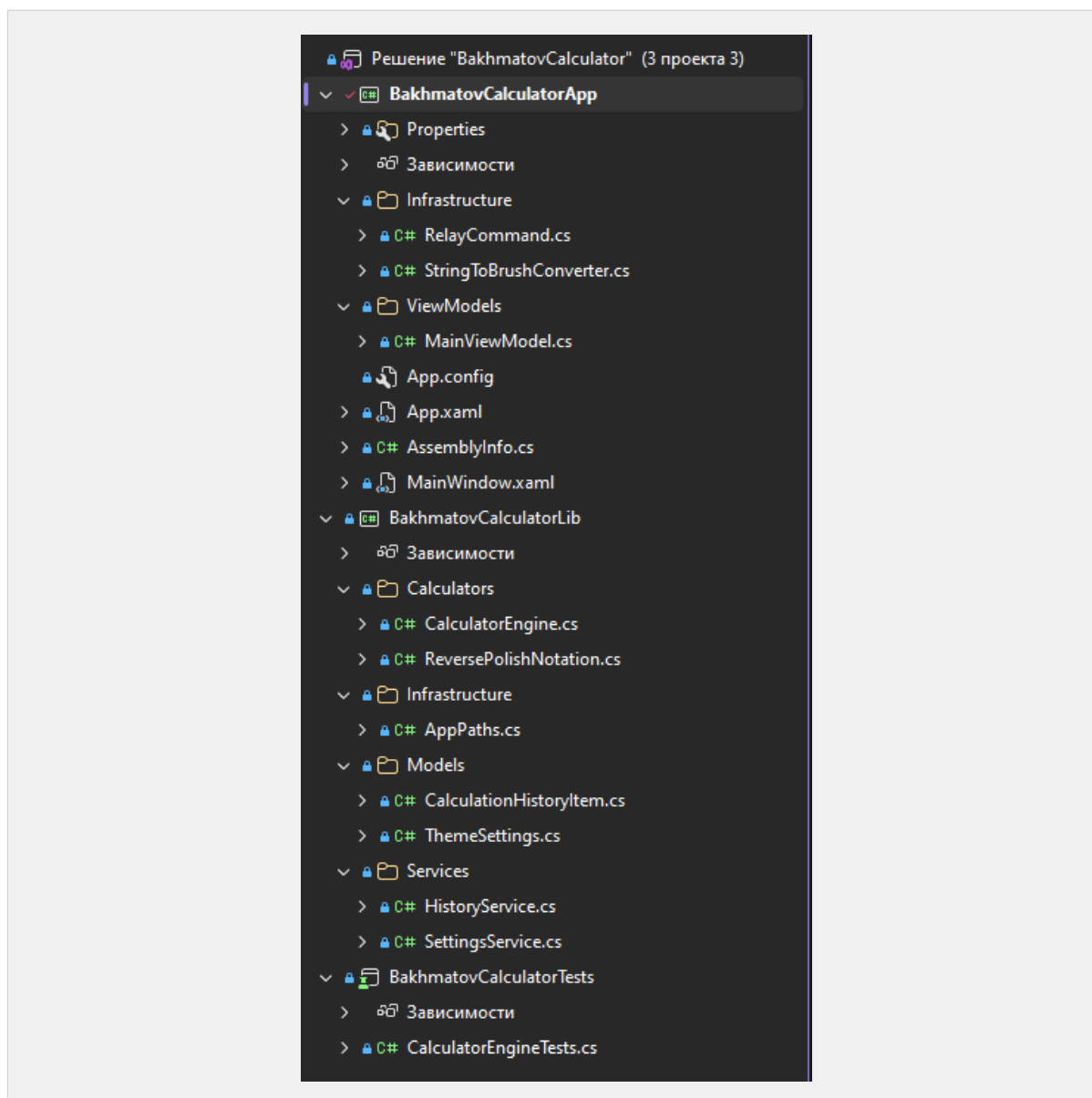


Рисунок 1 — Структура решения в Solution Explorer

Такое разделение соответствует принципу разделения ответственности: библиотека не зависит от WPF и может быть протестирована изолированно; приложение зависит только от библиотеки и не содержит бизнес-логики.

4. Создание проекта в Visual Studio

Для создания решения в Visual Studio 2022 был выполнен следующий порядок действий:

1. Создан новый Solution с именем BakhmatovCalculator
2. Добавлен проект Class Library (.NET 8) с именем BakhmatovCalculatorLib
3. Добавлен проект WPF Application (.NET 8) с именем BakhmatovCalculatorApp

4. Добавлен проект xUnit Test Project с именем BakhmatovCalculatorTests

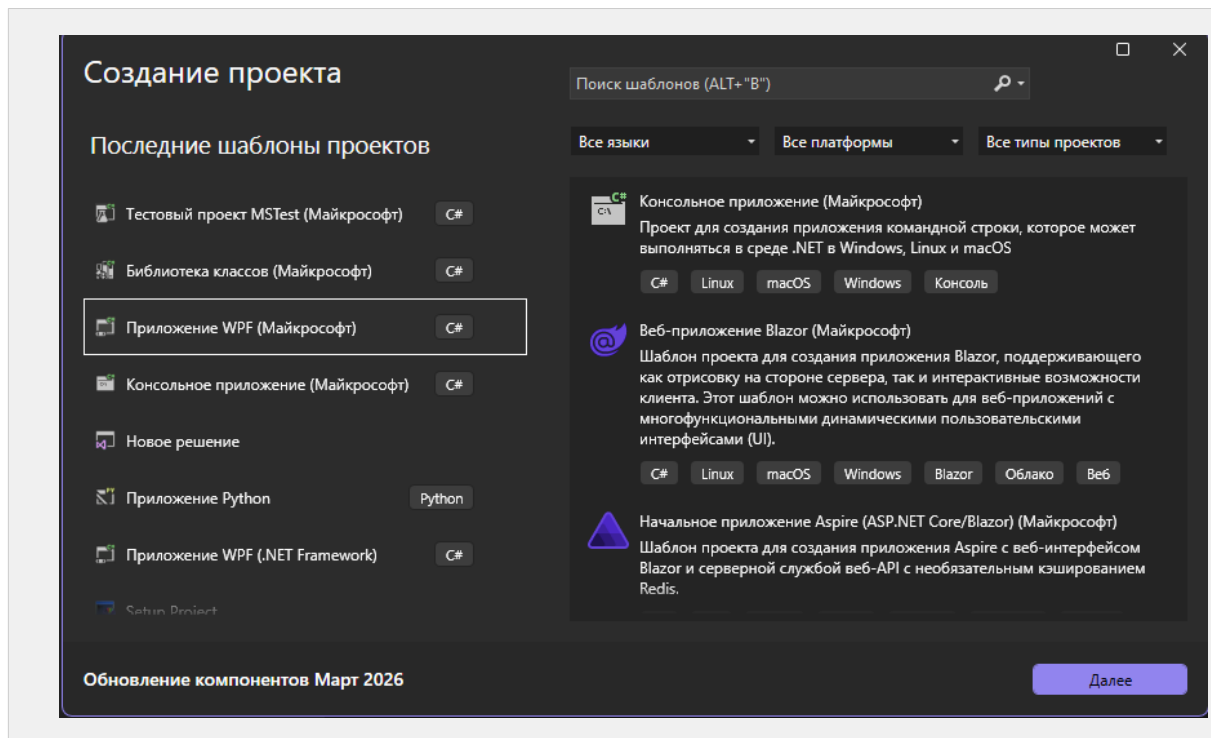


Рисунок 2 — Создание нового проекта в Visual Studio 2022

5. Реализация библиотеки классов

5.1 Алгоритм вычисления выражений

Для разбора и вычисления арифметических выражений реализован алгоритм Shunting-yard (алгоритм сортировочной станции Дейкстры). Алгоритм переводит инфиксную запись выражения в обратную польскую запись (RPN), после чего вычисляет результат через стек операндов.

Реализация разбита на три этапа:

5. Tokenize — токенизация строки: числа, операторы, скобки
6. ToRpn — перевод токенов в обратную польскую запись
7. EvaluateRpn — вычисление RPN-выражения через стек

```

// Примечание: унарный минус после ^ должен быть в скобках: 2^(-2), а не 2^-2.
// Shunting-yard выталкивает ^ из стека до того, как правый операнд попадает в него.
Ссылка: 1
private static IReadOnlyList<Token> ToRpn(IReadOnlyList<Token> tokens)
{
    var output = new List<Token>(tokens.Count);
    var opStack = new Stack<Token>();

    foreach (var token in tokens)
    {
        switch (token)
        {
            case NumberToken:
                output.Add(token);
                break;

            case OperatorToken opToken:
                while (opStack.Count > 0)
                {
                    var top = opStack.Peek();
                    if (top is not OperatorToken topOp)
                        break;

                    var (prec, assoc) = GetOperatorInfo(opToken.Kind);
                    var (topPrec, _) = GetOperatorInfo(topOp.Kind);

                    bool shouldPop =
                        assoc == OperatorAssociativity.Left
                        ? prec <= topPrec
                        : prec < topPrec;

                    if (!shouldPop)
                        break;

                    output.Add(opStack.Pop());
                }

                opStack.Push(opToken);
                break;

            case LeftParenToken:
                opStack.Push(token);
                break;

            case RightParenToken:
                while (opStack.Count > 0 && opStack.Peek() is not LeftParenToken)
                    output.Add(opStack.Pop());

                if (opStack.Count == 0)
                    throw new FormatException("Mismatched parentheses in expression.");

                opStack.Pop(); // remove '('
                break;

            default:
                throw new NotSupportedException("Unknown token.");
        }
    }

    while (opStack.Count > 0)
    {
        var token = opStack.Pop();
        if (token is LeftParenToken or RightParenToken)
            throw new FormatException("Mismatched parentheses in expression.");

        output.Add(token);
    }

    return output;
}

```

```

private static List<Token> Tokenize(string expression)
{
    var tokens = new List<Token>();

    int i = 0;
    Token? previousToken = null;

    while (i < expression.Length)
    {
        var c = expression[i];

        if (char.IsWhiteSpace(c))
        {
            i++;
            continue;
        }

        if (c == '(')
        {
            tokens.Add(new LeftParenToken());
            previousToken = tokens[^1];
            i++;
            continue;
        }

        if (c == ')')
        {
            tokens.Add(new RightParenToken());
            previousToken = tokens[^1];
            i++;
            continue;
        }

        if (IsBinaryOperatorChar(c) || c == '-')
        {
            if (c == '-')
            {
                bool unary = previousToken is null
                    || previousToken is OperatorToken
                    || previousToken is LeftParenToken;

                tokens.Add(new OperatorToken(unary ? OperatorKind.UnaryMinus : OperatorKind.Subtract));
            }
            else
            {
                tokens.Add(new OperatorToken(ToOperatorKind(c)));
            }

            previousToken = tokens[^1];
            i++;
            continue;
        }

        if (char.IsDigit(c) || c == '.' || c == ',')
        {
            int start = i;
            bool seenSeparator = false;

            i++; // consume first char
            while (i < expression.Length)
            {
                char ch = expression[i];
                if (char.IsDigit(ch))
                {
                    i++;
                    continue;
                }

                if ((ch == '.' || ch == ',') && !seenSeparator)
                {
                    seenSeparator = true;
                    i++;
                    continue;
                }

                break;
            }

            var numberText = expression.Substring(start, i - start).Replace(',', '.');
            if (numberText == "." || numberText == "-." || numberText == "-,")
                throw new FormatException("Invalid number format.");

            if (!decimal.TryParse(numberText, NumberStyles.Number, CultureInfo.InvariantCulture, out var value))
                throw new FormatException($"Invalid number: '{numberText}'");

            tokens.Add(new NumberToken(value));
            previousToken = tokens[^1];
            continue;
        }

        throw new FormatException($"Unexpected character '{c}' in expression.");
    }
}

```

```
if (tokens.Count == 0)
    throw new FormatException("Expression is empty.");

return tokens;
}
```

Рисунок 3 — Реализация алгоритма Shunting-yard в ReversePolishNotation.cs

Для хранения чисел используется тип decimal вместо double, что исключает ошибки представления с плавающей точкой (например, $0.1 + 0.2 = 0.3$, а не 0.30000000000000004). Вся арифметика выполняется в блоке checked — переполнение вызывает OverflowException.

5.2 CalculatorEngine

Класс CalculatorEngine является фасадом библиотеки и предоставляет следующий API:

Метод	Описание
Calculate(string)	Вычисление и запись в историю
Evaluate(string)	Вычисление без записи в историю (предпросмотр)
GetHistory()	Список истории (до 200 записей)
ClearHistory()	Очистка истории с сохранением
GetThemeSettings()	Получение настроек темы
UpdateThemeSettings()	Обновление и сохранение темы
SaveAll()	Явное сохранение на диск

```

Ссылка 11
public sealed class CalculatorEngine
{
    private const int MaxHistoryItems = 200;

    private readonly HistoryService _historyService;
    private readonly SettingsService _settingsService;

    private readonly List<CalculationHistoryItem> _history;
    private ThemeSettings _settings;

    Ссылка 8 | 0/3 passing
    public CalculatorEngine(string? historyFilePath = null, string? settingsFilePath = null)
    {
        _historyService = new HistoryService(historyFilePath);
        _settingsService = new SettingsService(settingsFilePath);

        _history = new List<CalculationHistoryItem>(_historyService.Load());
        _settings = _settingsService.Load();
    }

    /// <summary>
    /// Вычисляет выражение и сохраняет результат в историю.
    /// </summary>
    Ссылка 16 | 0/37 passing
    public decimal Calculate(string expression)
    {
        var normalized = NormalizeExpression(expression);
        var result = ReversePolishNotation.Evaluate(normalized);
        AddHistory(normalized, result);
        return result;
    }

    /// <summary>
    /// Вычисляет выражение без записи в историю (preview).
    /// </summary>
    Ссылка 3 | 0/1 passing
    public decimal Evaluate(string expression)
    {
        var normalized = NormalizeExpression(expression);
        return ReversePolishNotation.Evaluate(normalized);
    }

    Ссылка 8 | 0/6 passing
    public IReadOnlyList<CalculationHistoryItem> GetHistory() => _history;

    Ссылка 3 | 0/2 passing
    public void ClearHistory()
    {
        Ссылка 3 | 0/2 passing
        public void ClearHistory()
        {
            _history.Clear();
            SaveAll();
        }

        Ссылка 5 | 0/3 passing
        public ThemeSettings GetThemeSettings() => _settings;

        Ссылка 4 | 0/3 passing
        public void UpdateThemeSettings(ThemeSettings settings)
        {
            _settings = settings ?? throw new ArgumentNullException(nameof(settings));
            SaveAll();
        }

        Ссылка 4 | 0/1 passing
        public void SaveAll()
        {
            _historyService.Save(_history);
            _settingsService.Save(_settings);
        }

        Ссылка 1
        private void AddHistory(string expression, decimal result)
        {
            _history.Add(new CalculationHistoryItem(expression, result, DateTime.UtcNow));

            if (_history.Count > MaxHistoryItems)
                _history.RemoveRange(0, _history.Count - MaxHistoryItems);
        }

        Ссылка 2
        private static string NormalizeExpression(string expression)
        => expression?.Trim() ?? throw new ArgumentNullException(nameof(expression));
    }
}

```

Рисунок 4 — Класс CalculatorEngine

5.3 Модели

Для хранения данных используются две модели:

- CalculationHistoryItem — иммутабельная запись истории (Expression, Result, Timestamp). Сериализуется через [JsonConstructor] без публичных сеттеров.
- ThemeSettings — настройки темы (IsDarkTheme, AccentColor). Содержит статическое поле Default с настройками по умолчанию.

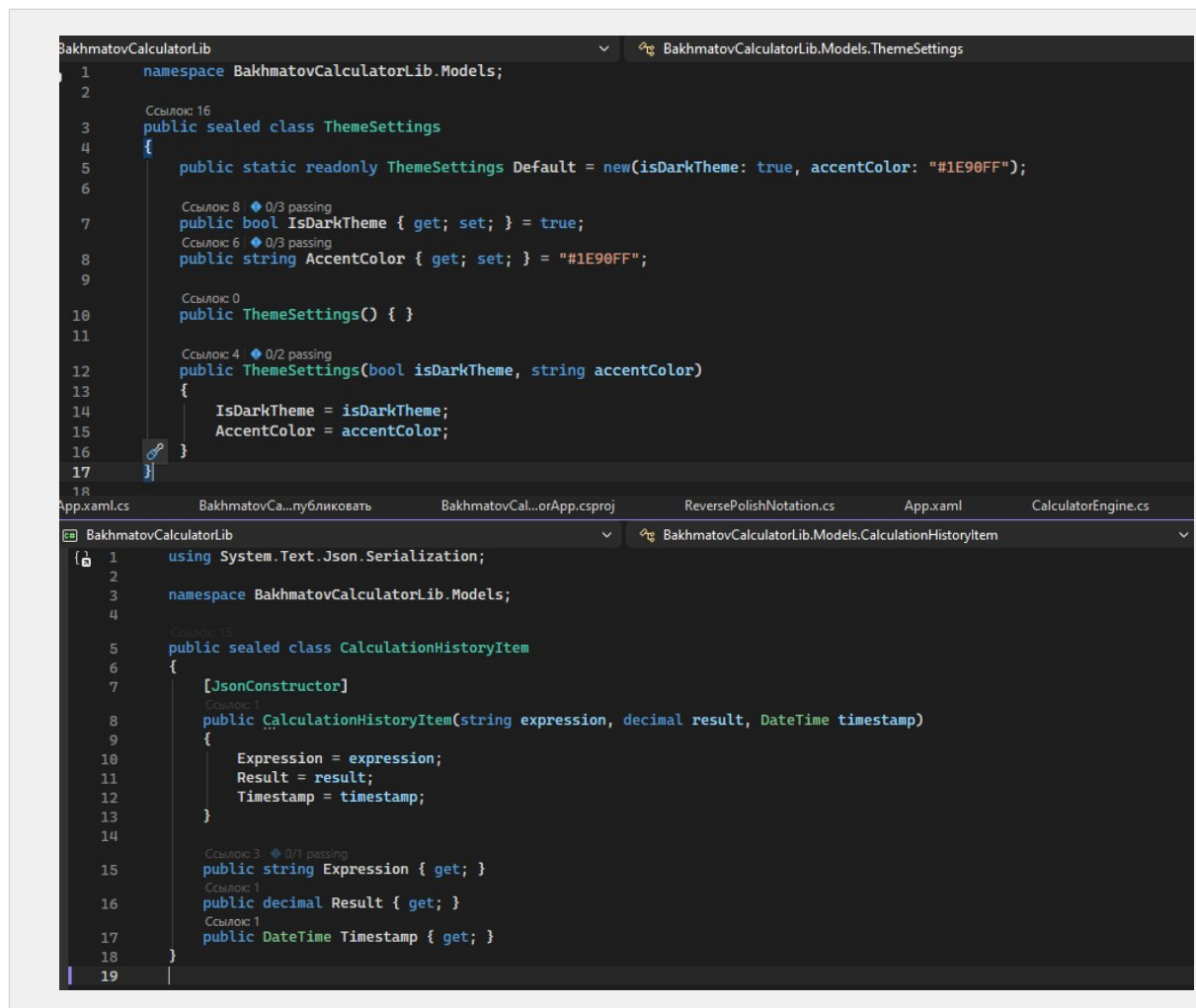


Рисунок 5 — Модели данных

5.4 Сервисы

Для персистентности данных реализованы два сервиса:

- HistoryService — сохранение и загрузка истории в history.json
- SettingsService — сохранение и загрузка настроек в settings.json

Файлы хранятся в %LocalAppData%\BakhmatovCalculator\. Для устранения дублирования пути создан вспомогательный класс AppPaths. JsonSerializerOptions вынесены в статические поля для исключения лишних аллокаций.

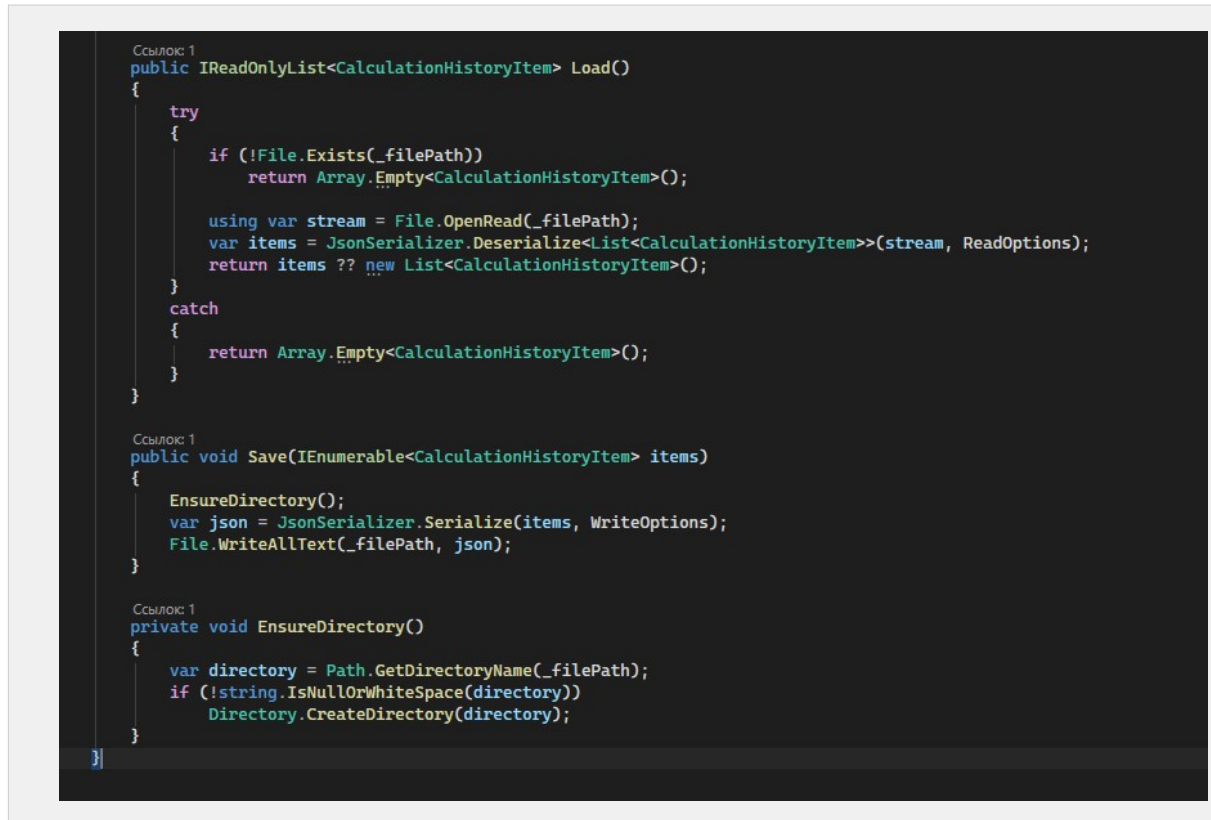


Рисунок 6 — Сервис работы с историей

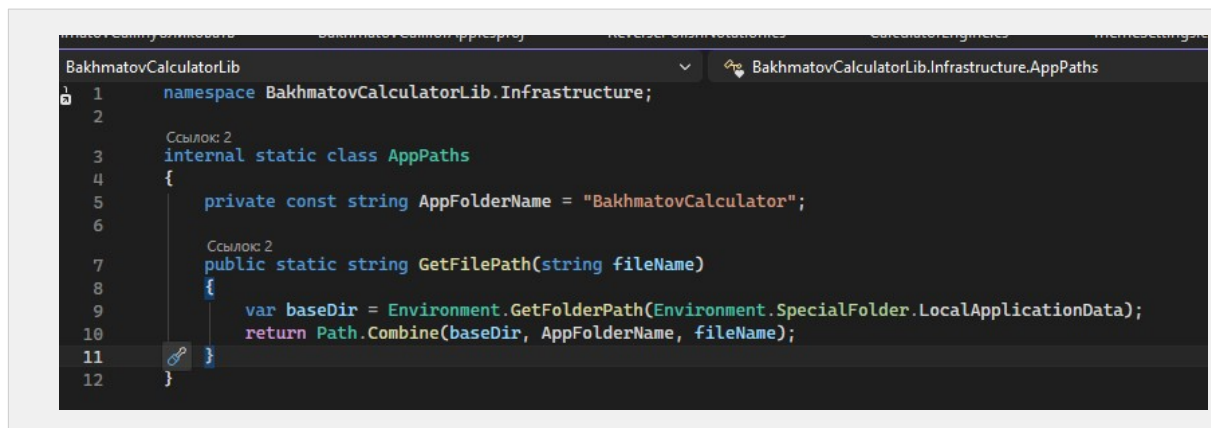


Рисунок 7 — Класс AppPaths

6. Реализация WPF-приложения

6.1 Архитектура MVVM

Приложение построено по паттерну MVVM (Model-View-ViewModel):

- Model — классы библиотеки BakhmatovCalculatorLib
- ViewModel — MainViewModel, реализует INotifyPropertyChanged
- View — MainWindow.xaml с привязками данных

ViewModel не содержит зависимостей на System.Windows.Media — цвета темы хранятся как HEX-строки и преобразуются в Brush только на стороне View через конвертер StringToBrushConverter.

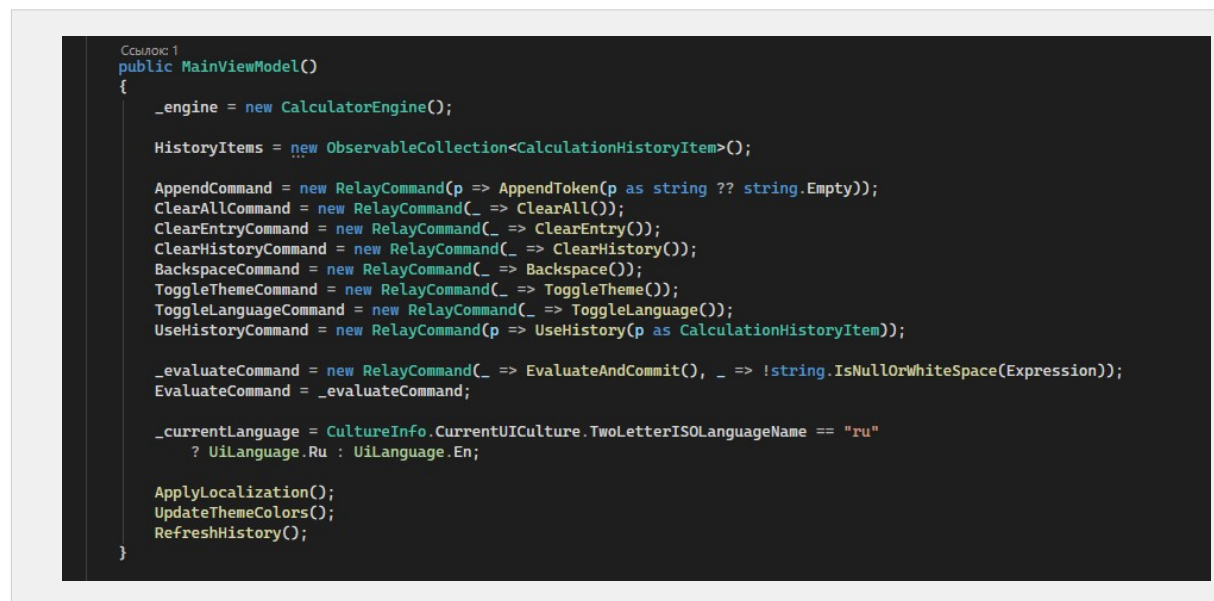


Рисунок 8 — MainViewModel: инициализация команд

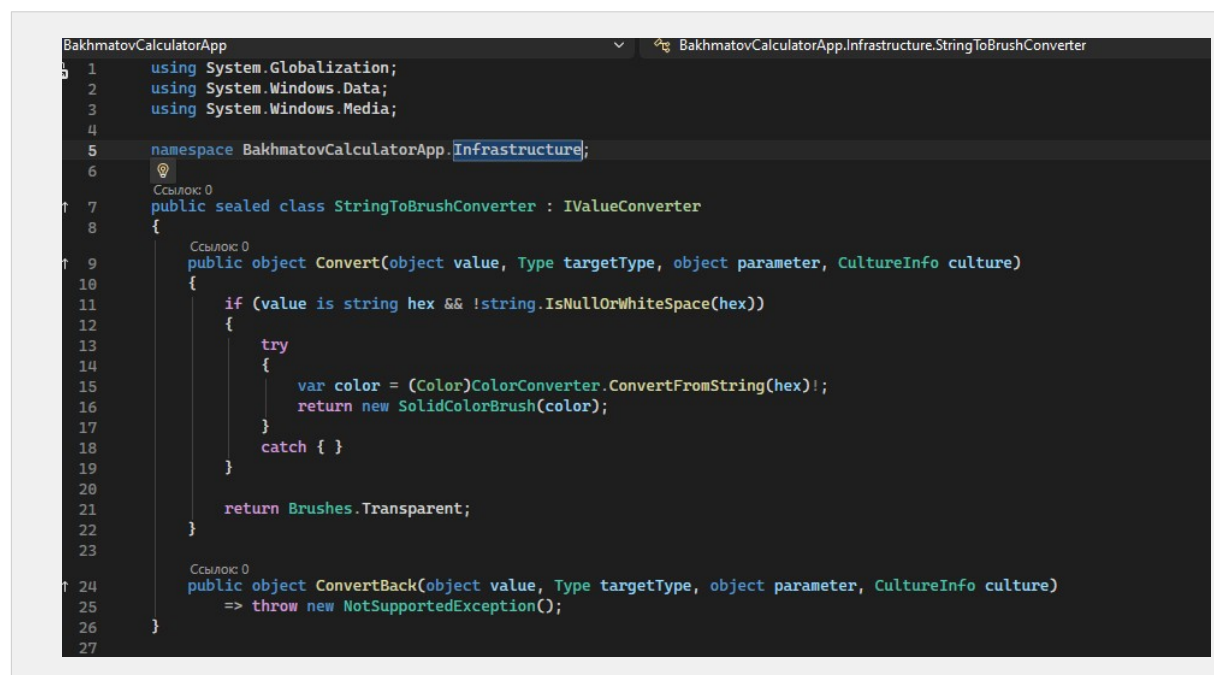


Рисунок 9 — Конвертер StringToBrushConverter

6.2 Интерфейс

Дизайн приложения выполнен в стиле Windows 11 Calculator. Интерфейс разделён на две колонки: основная с дисплеем и клавиатурой, и боковая панель истории.

Клавиатура имеет иерархию стилей кнопок:

- FuncButtonStyle — служебные кнопки: (,), CE, C, x^y, ☒
- NumButtonStyle — цифры 0–9 и десятичная точка
- OpButtonStyle — операторы ÷ × − +
- EqualButtonStyle — кнопка =

Кнопки имеют анимацию нажатия (scale) через WPF Storyboard с QuadraticEase.

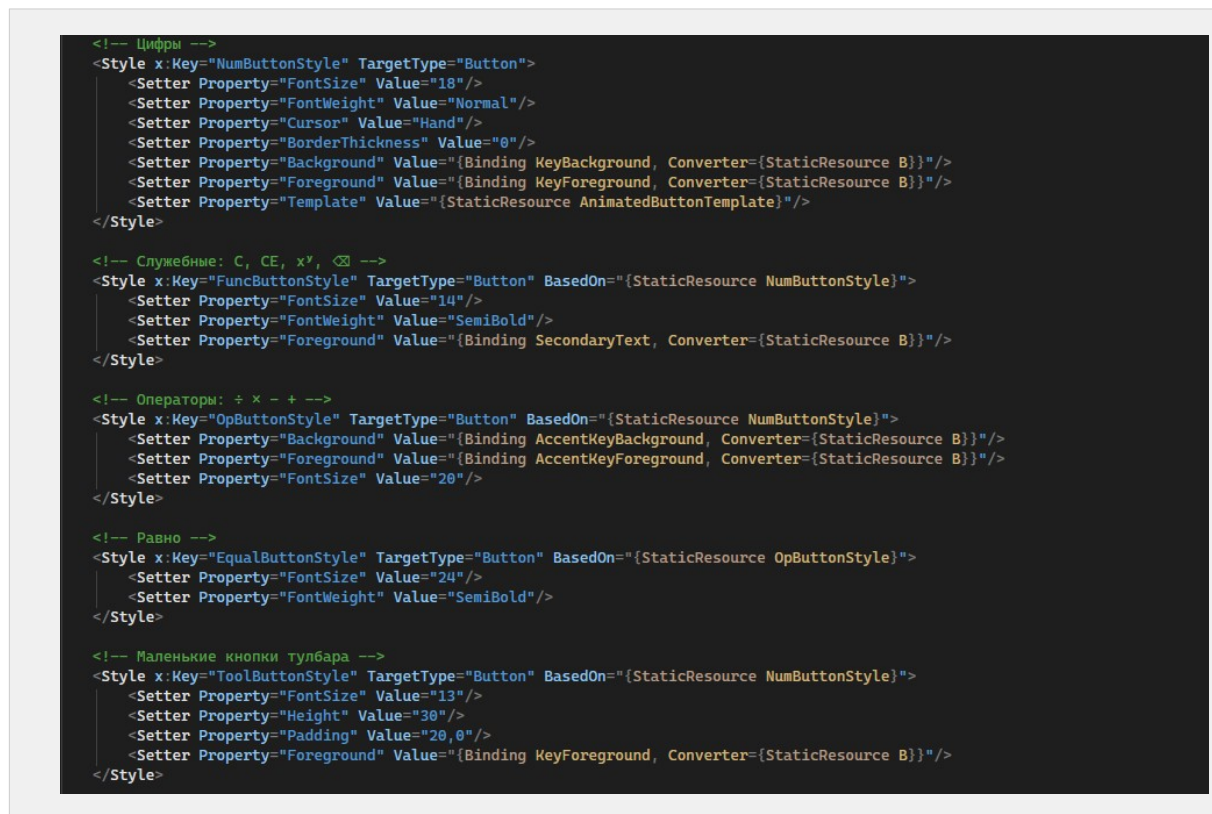


Рисунок 10 — Стили кнопок в MainWindow.xaml

6.3 Возможности приложения

Приложение поддерживает следующие возможности:

- Вычисление арифметических выражений с операторами +, −, *, /, ^ и скобками
- Унарный минус в начале выражения и после оператора
- Предпросмотр результата в реальном времени без нажатия =
- История вычислений с возможностью восстановить выражение кликом
- Очистка истории
- Переключение тёмной и светлой темы с сохранением настройки
- Локализация интерфейса RU / EN

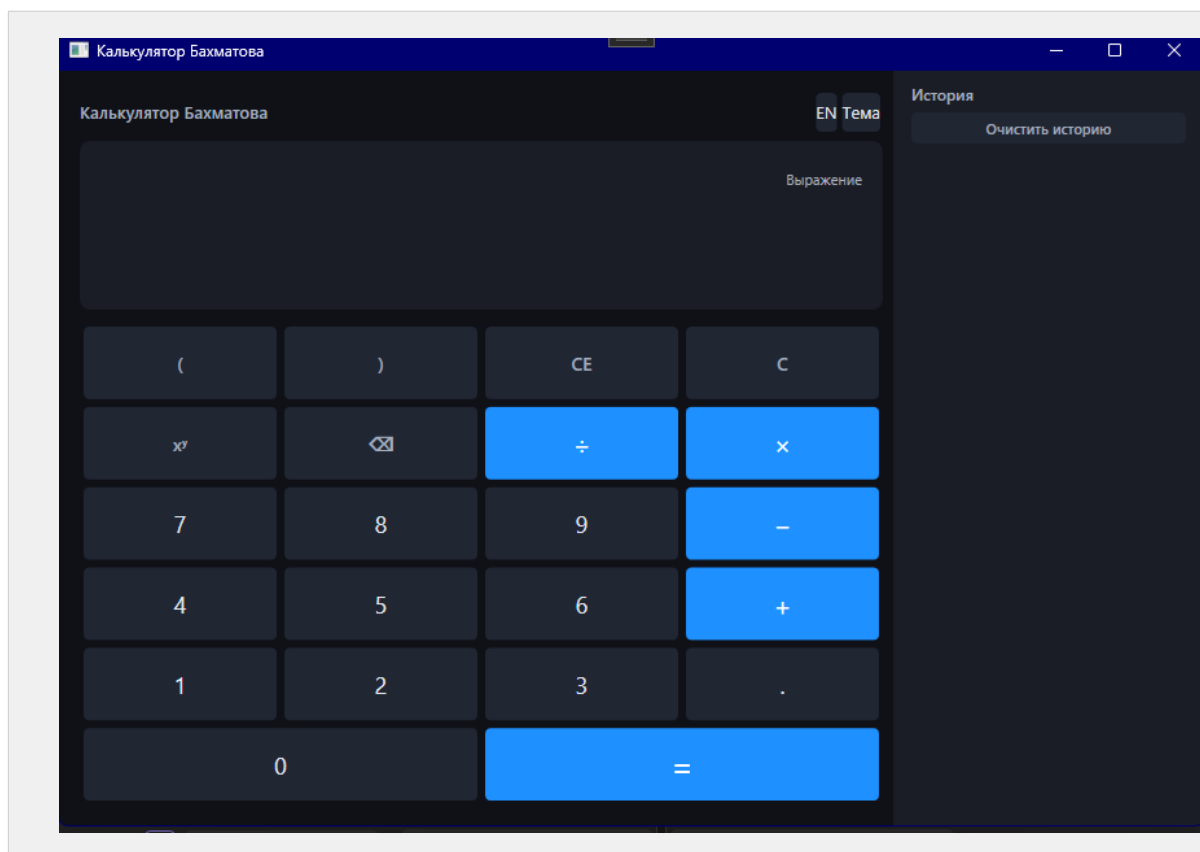


Рисунок 11 — Приложение в тёмной теме

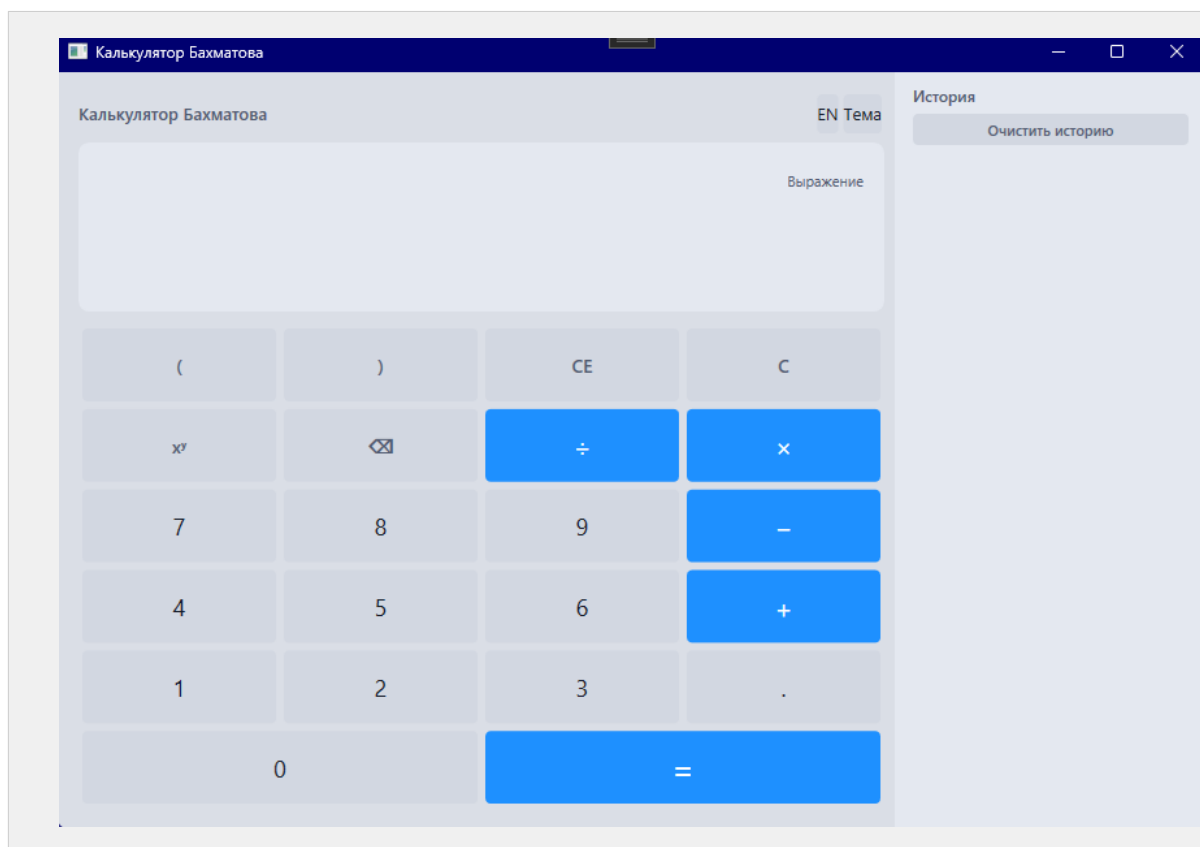


Рисунок 12 — Приложение в светлой теме

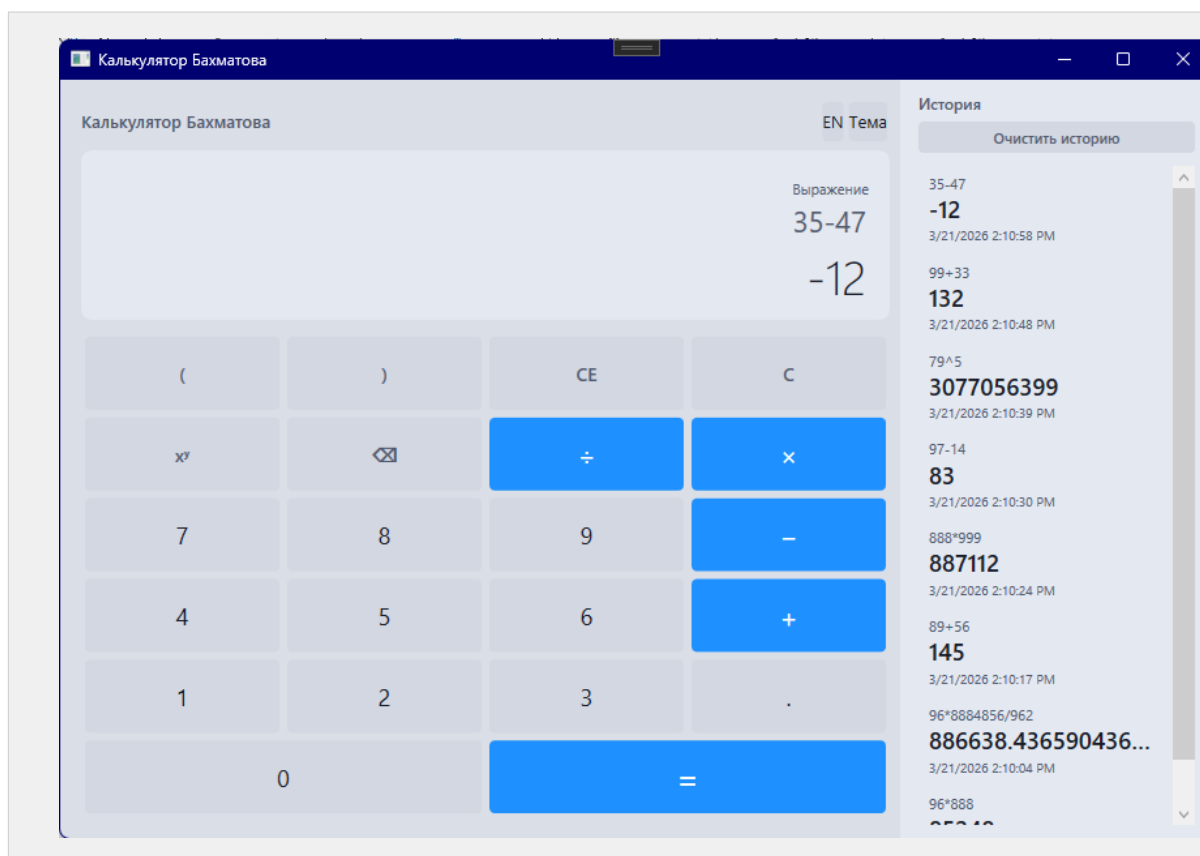


Рисунок 13 — Панель истории вычислений

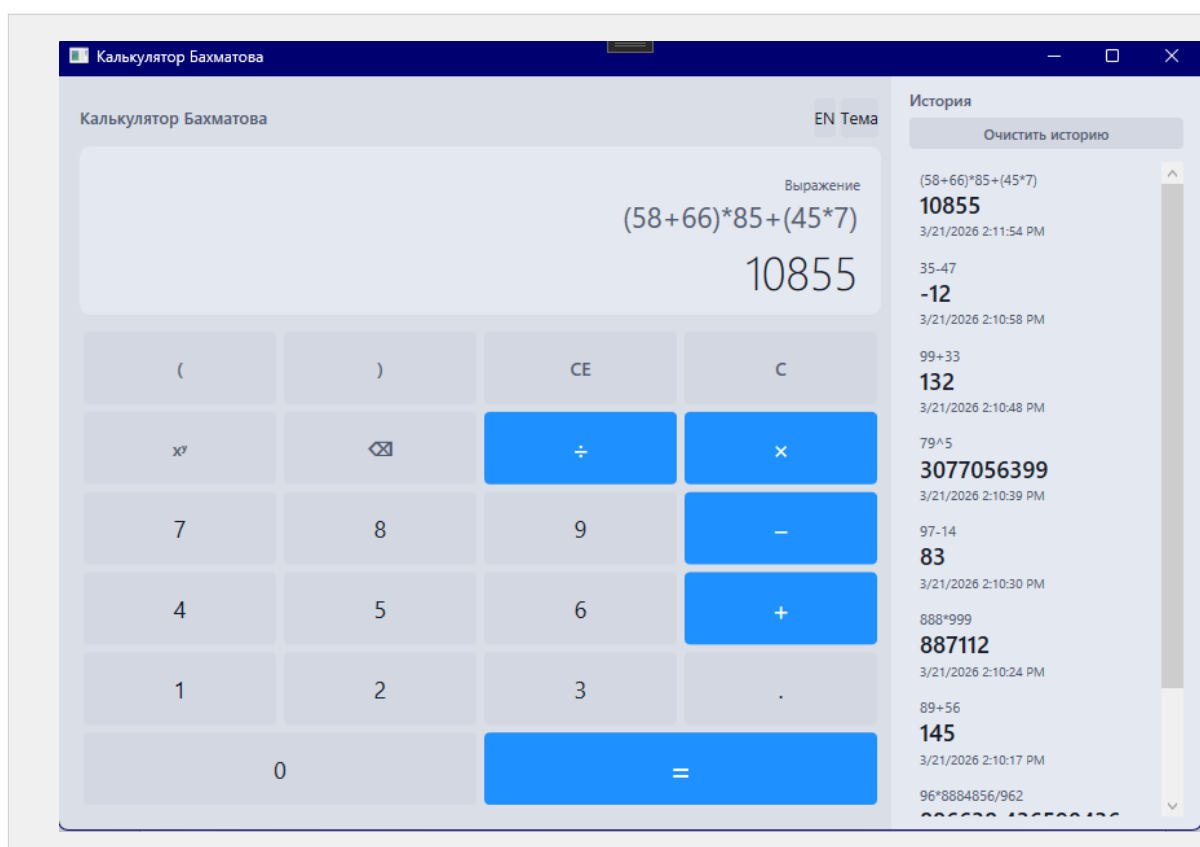


Рисунок 14 — Вычисление выражения со скобками

6.4 Ввод с клавиатуры

Помимо мыши, поддерживается ввод с клавиатуры:

Клавиша	Действие
Цифры, операторы	Ввод символа в выражение
Точка / запятая	Десятичный разделитель
(и)	Скобки
Enter / Return	Вычисление результата
Backspace	Удаление последнего символа
Escape	Полная очистка выражения

6.5 Сохранение данных

При закрытии окна вызывается `MainViewModel.Persist()`, который через `CalculatorEngine.SaveAll()` сохраняет историю и настройки темы в JSON-файлы:

- `%LocalAppData%\BakhmatovCalculator\history.json`
- `%LocalAppData%\BakhmatovCalculator\settings.json`

7. Модульное тестирование

Для библиотеки `BakhmatovCalculatorLib` написаны модульные тесты с использованием фреймворка `xUnit`. Тесты охватывают все основные сценарии работы приложения.

7.1 Структура тестов

Тесты организованы в группы с использованием атрибутов `[Fact]` для одиночных тестов и `[Theory] + [InlineData]` для параметризованных:

Группа тестов	Что проверяется
Базовые операции	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>^</code> через <code>[Theory]</code>
Десятичные числа	Точка и запятая, точность <code>decimal</code>
Приоритет операций	<code>*</code> , <code>/</code> , <code>^</code> перед <code>+</code> , <code>-</code> , скобки, ассоциативность
Унарный минус	В начале, после оператора, в скобках
Степень	<code>5^0</code> , <code>5^1</code> , <code>2^(-2)</code> , дробный показатель
Негативные кейсы	Деление на ноль, переполнение, ошибки формата

История	Добавление, сар на 200 записей, персистентность
Настройки темы	Получение, обновление, персистентность, null

```

[Theory]
[InlineData("2+3", 5)]
[InlineData("10-4", 6)]
[InlineData("7*8", 56)]
[InlineData("20/5", 4)]
[InlineData("2^10", 1024)]
// Ссылка 0
public void Calculate_BasicOperations_ReturnCorrectResult(string expression, decimal expected)
{
    var engine = CreateEngine();
    Assert.Equal(expected, engine.Calculate(expression));
}

[Theory]
[InlineData("1.5+2.5", 4)]
[InlineData("0.1+0.2", 0.3)]
[InlineData("10.0/4.0", 2.5)]
[InlineData("1.5+2.5", 4)] // запятая как разделитель
[InlineData("3,14*2", 6.28)]
// Ссылка 0
public void Calculate_DecimalNumbers_ReturnCorrectResult(string expression, decimal expected)
{
    var engine = CreateEngine();
    Assert.Equal(expected, engine.Calculate(expression));
}

[Theory]
[InlineData("2+3*4", 14)] // * перед +
[InlineData("(2+3)*4", 20)] // скобки меняет порядок
[InlineData("2^3^2", 512)] // ^ правоассоциативен: 2^(3^2) = 2^9
[InlineData("10-2-3", 5)] // - левоассоциативен: (10-2)-3
[InlineData("2*3+4*5", 26)]
// Ссылка 0
public void Calculate_PrecedenceAndAssociativity_IsCorrect(string expression, decimal expected)
{
    var engine = CreateEngine();
    Assert.Equal(expected, engine.Calculate(expression));
}

[Theory]
[InlineData("-5+2", -3)]
[InlineData("3*-2", -6)]
[InlineData("(-5)*(-2)", 10)]
[InlineData("-2^2", -4)] // -(2^2), а не (-2)^2
// Ссылка 0
public void Calculate_UnaryMinus_IsHandledCorrectly(string expression, decimal expected)
{
    var engine = CreateEngine();
    Assert.Equal(expected, engine.Calculate(expression));
}

[Theory]
[InlineData("5^0", 1)]
[InlineData("5^1", 5)]
[InlineData("2^(-2)", 0.25)] // отрицательный показатель
// Ссылка 0
public void Calculate_Power_EdgeCases(string expression, decimal expected)
{
    var engine = CreateEngine();
    Assert.Equal(expected, engine.Calculate(expression));
}

[Theory]
[InlineData("(2+3)"] // незакрытая скобка
[InlineData("2+3)"] // лишняя закрывающая
[InlineData("")] // пустое выражение
[InlineData(" ")] // только пробелы
[InlineData("2@3")] // недопустимый символ
[InlineData("2++3")] // два оператора подряд
[InlineData("+")] // только оператор
// Ссылка 0
public void Calculate_InvalidExpression_ThrowsFormatException(string expression)
{
    var engine = CreateEngine();
    Assert.Throws<FormatException>(() => engine.Calculate(expression));
}

```

Рисунок 15 — Параметризованные тесты с [Theory] и [InlineData]

7.2 Изоляция тестов

Каждый тест создаёт изолированный экземпляр CalculatorEngine с временными файлами в уникальной папке в %TEMP%:



Рисунок 16 — Метод `CreateEngine()` — изоляция тестов через временные файлы

7.3 Результаты тестирования

Все тесты успешно проходят. Запуск осуществляется через Test Explorer в Visual Studio или командой:

```
dotnet test BakhmatovCalculator.sln -c Release
```

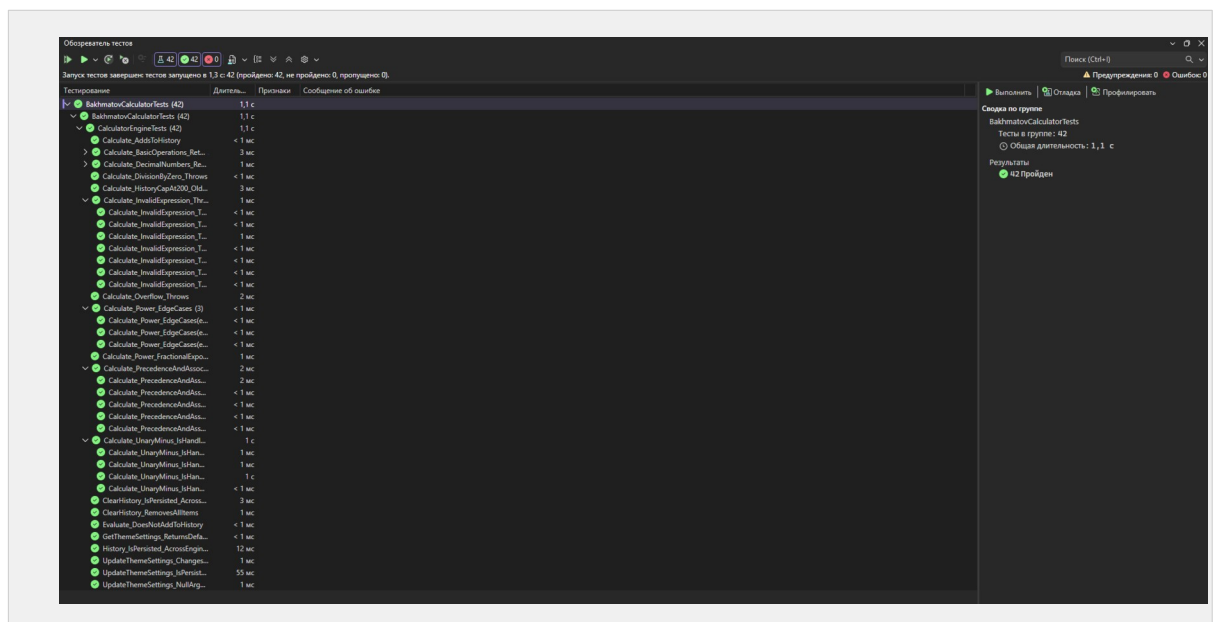


Рисунок 17 — Результаты выполнения тестов в Test Explorer

```

Перестроение начато в 19:20...
Восстановлен E:\BakhmatovCalculator\BakhmatovCalculatorLib\BakhmatovCalculatorLib.csproj (за 2 мс).
Восстановлен E:\BakhmatovCalculator\BakhmatovCalculatorApp\BakhmatovCalculatorApp.csproj (за 3 мс).
Восстановлен E:\BakhmatovCalculator\BakhmatovCalculatorTests\BakhmatovCalculatorTests.csproj (за 10,34 с).
1>----- Перестроение всех файлов начато: проект: BakhmatovCalculatorLib, Конфигурация: Debug Any CPU -----
1> BakhmatovCalculatorLib -> E:\BakhmatovCalculator\BakhmatovCalculatorLib\bin\Debug\net8.0\BakhmatovCalculatorLib.dll
2>----- Перестроение всех файлов начато: проект: BakhmatovCalculatorApp, Конфигурация: Debug Any CPU -----
3>----- Перестроение всех файлов начато: проект: BakhmatovCalculatorTests, Конфигурация: Debug Any CPU -----
3> BakhmatovCalculatorTests -> E:\BakhmatovCalculator\BakhmatovCalculatorTests\bin\Debug\net8.0\BakhmatovCalculatorTests.dll
2> BakhmatovCalculatorApp -> E:\BakhmatovCalculator\BakhmatovCalculatorApp\bin\Debug\net8.0-windows\BakhmatovCalculatorApp.dll
===== Перестроить все: успешно - 3, неудачно - 0, пропущено - 0 =====
===== Перестроить завершено в 19:20 и заняло 16,856 с =====

```

Рисунок 18 — Успешная сборка решения

8. Публикация приложения

Приложение опубликовано как self-contained single-file исполняемый файл для платформы Windows x64. Это означает, что для запуска не требуется установка .NET Runtime на целевом компьютере.

Параметры публикации:

- Configuration: Release
- Target framework: net8.0-windows
- Deployment mode: Self-contained
- Target runtime: win-x64
- Produce single file: включено

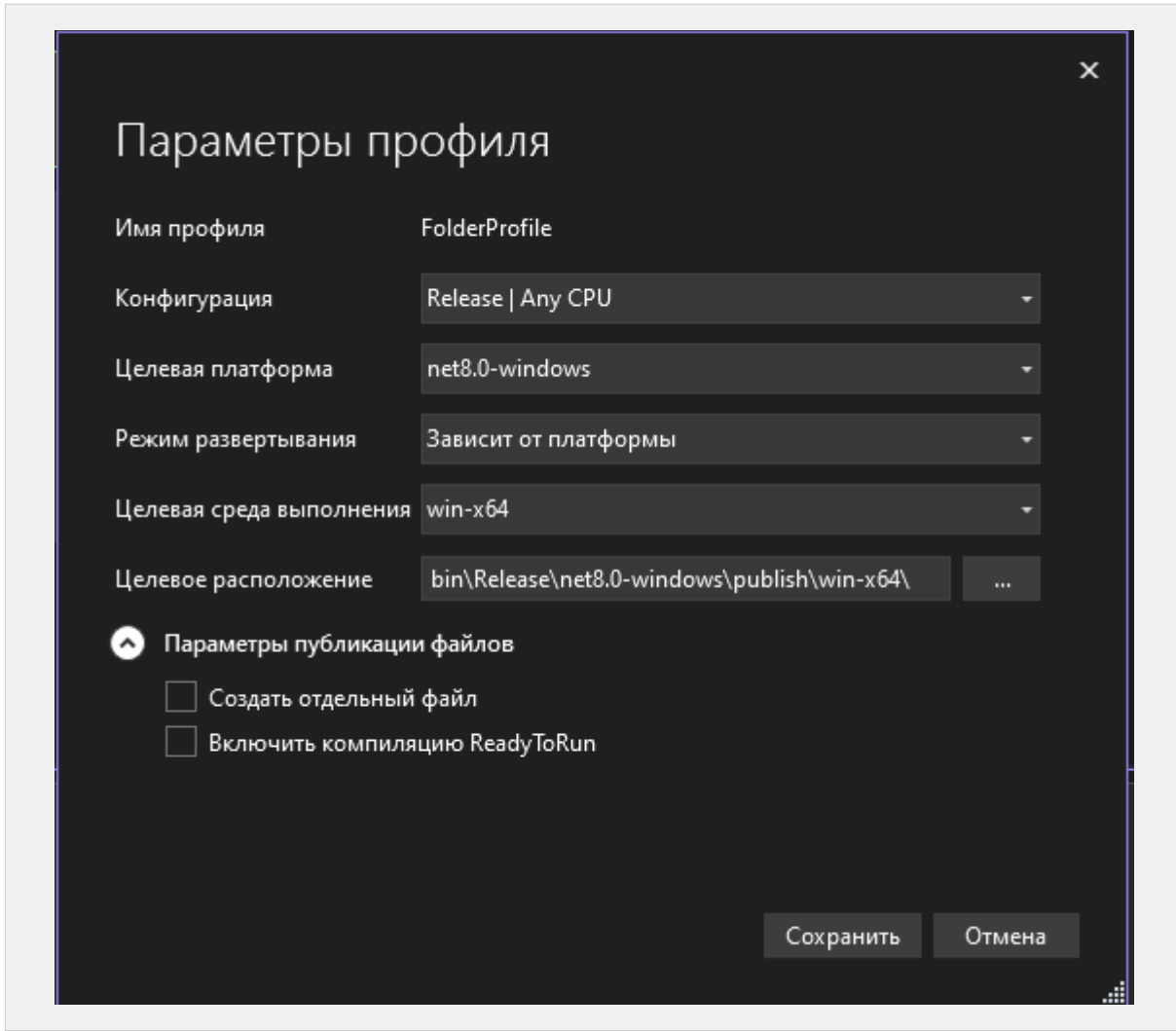


Рисунок 19 — Параметры публикации в Visual Studio

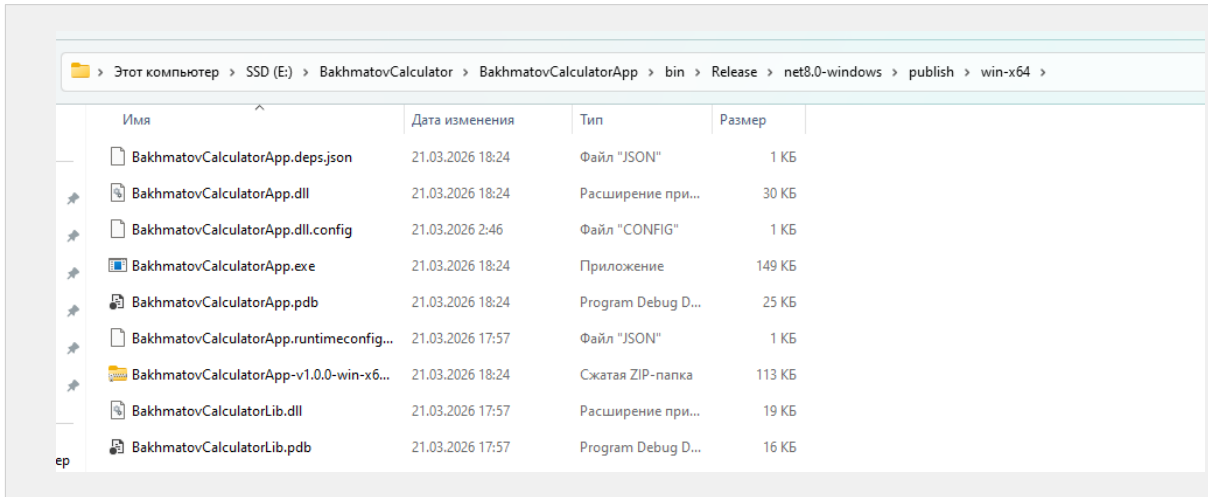


Рисунок 20 — Содержимое папки publish после публикации

9. Размещение на GitHub

Исходный код проекта размещён в публичном репозитории на GitHub по адресу: <https://github.com/Linkimin/Calculator>

Репозиторий содержит:

- Исходный код всех трёх проектов
- README.md с описанием проекта и инструкцией по запуску
- Report.md — техническое описание решения
- LICENSE — лицензия MIT
- Release v1.0.0 с готовым исполняемым файлом для Windows x64

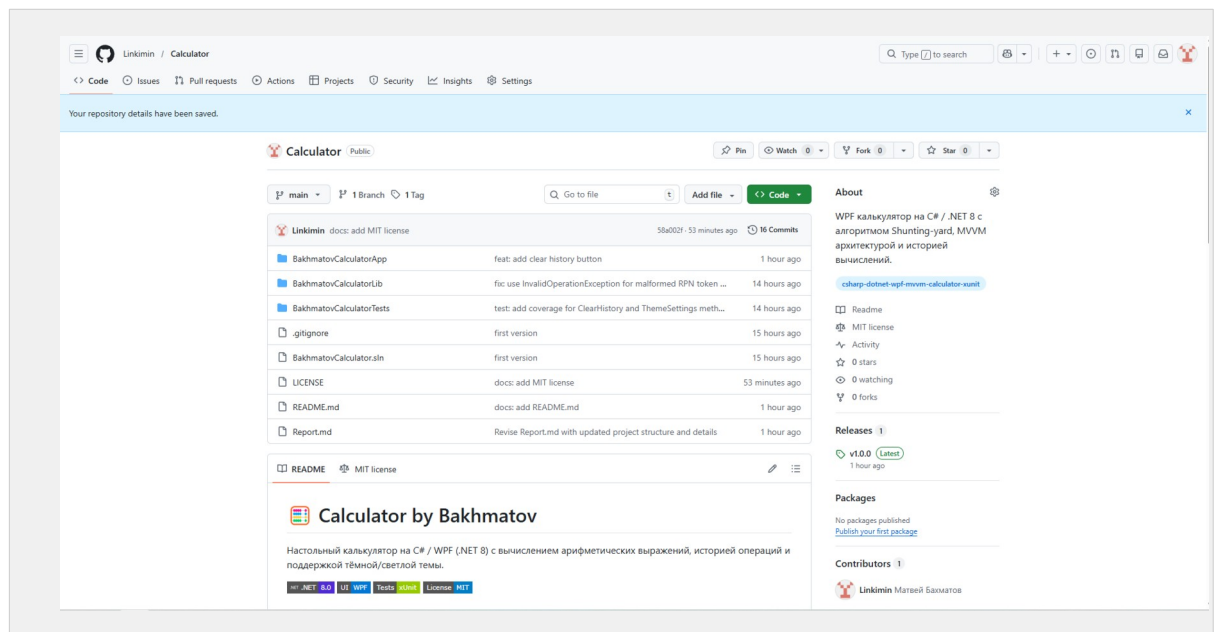


Рисунок 21 — Репозиторий на GitHub

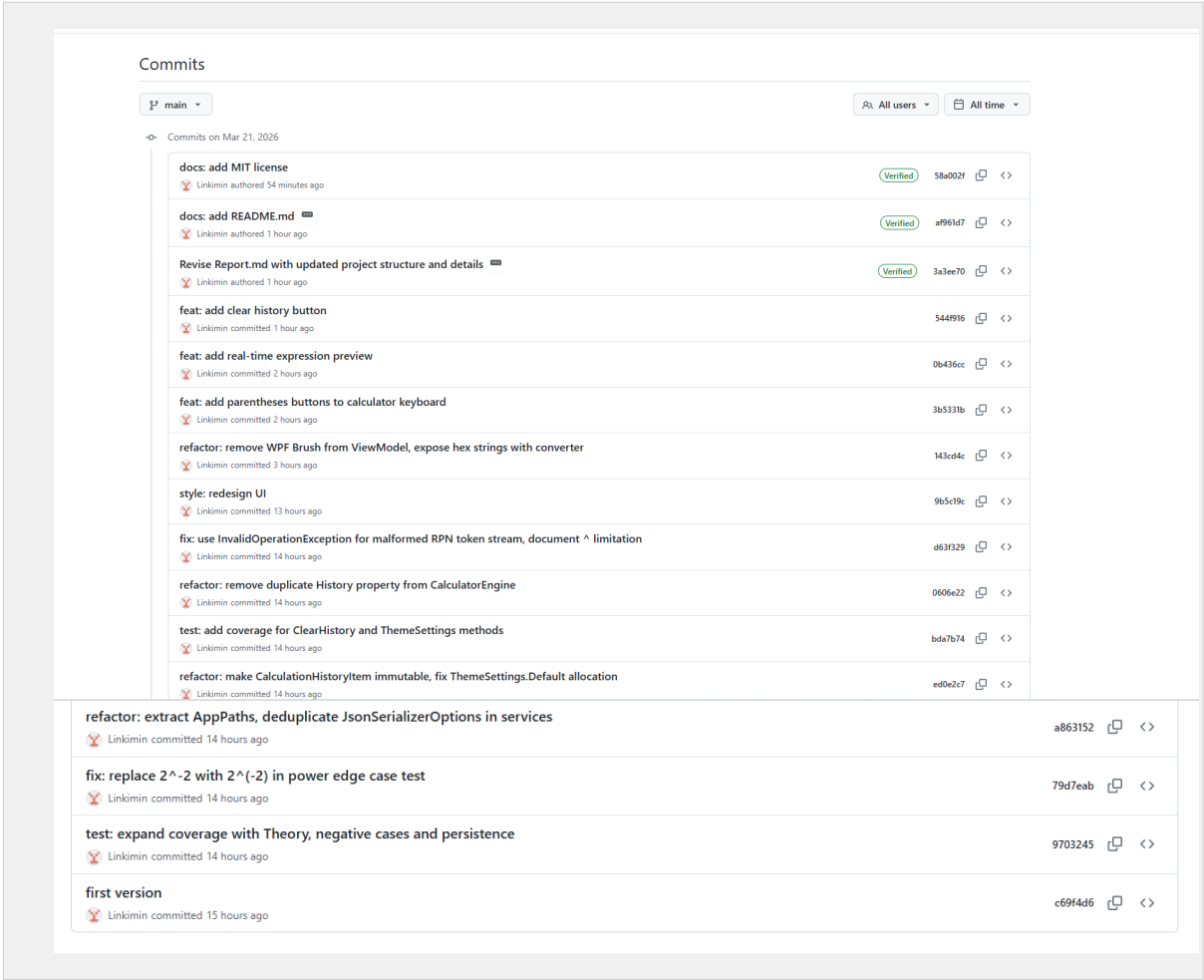


Рисунок 22 — История коммитов

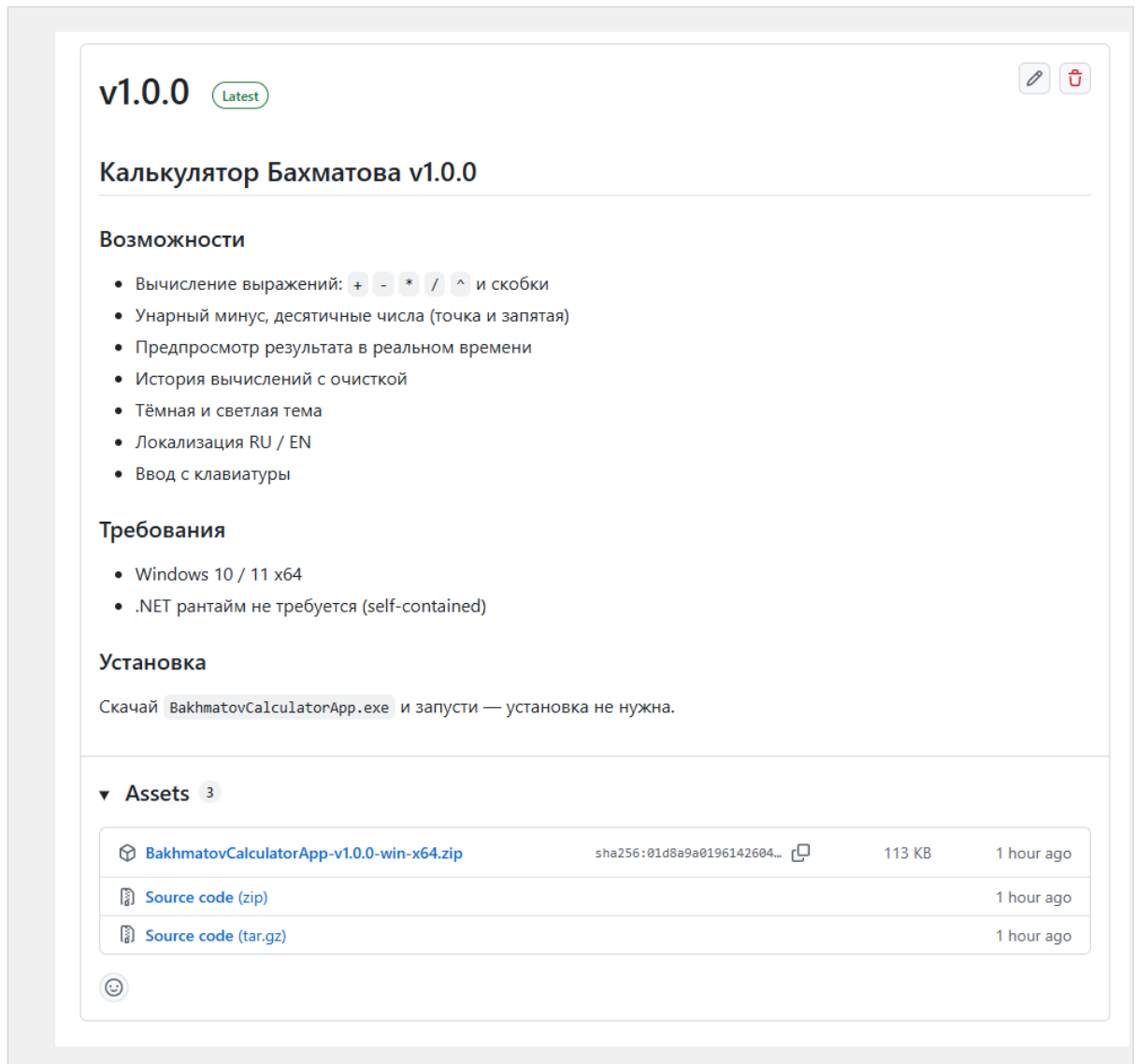


Рисунок 23 — Release v1.0.0 с готовым билдом

10. Заключение

В ходе практической работы было разработано настольное приложение «Калькулятор» на платформе .NET 8 с использованием технологии WPF. Все требования задания выполнены:

- Реализованы арифметические операторы +, -, *, /, ^ и скобки
- Логика вынесена в отдельную библиотеку классов `BakhmatovCalculatorLib`
- Интерфейс реализован в виде WPF-приложения по паттерну MVVM
- Написаны модульные тесты с использованием xUnit (25+ тестов)
- Название проекта содержит фамилию студента: `BakhmatovCalculatorApp`
- Исходный код и сопроводительная записка размещены на GitHub

Дополнительно реализованы: история вычислений, тёмная/светлая тема, локализация RU/EN, предпросмотр результата в реальном времени, ввод с клавиатуры. Приложение опубликовано как self-contained исполняемый файл для Windows x64.